There is typically a "safe" version of the same method that does not end in ! and does not modify the receiver or raise an exception.

Ruby style tends to avoid explicitly using self as the receiver of a method unless it is required to avoid ambiguity, as in the left side of an assignment. Ruby style also avoids explicitly and redundantly using return on the last line of a method where the return value is already specified. Only explicitly use return when exiting early from a method.

Single-line method calls should only be used when the body of the method is a single expression and has no side effects. Ideally, it also has no arguments. For example:

```
def price_in_dollars = price_in_cents * 1.0 / 100.
```

Also, you should use attr_reader, attr_writer, and attr_accessor when appropriate rather than explicitly defining simple setters and getters.

Ruby allows parallel assignment between multiple left and right arguments. In general, use parallel assignment to either:

- swap values: first, second = second, first or
- capture multiple return values from a method: user, count = post.user_with_most_comments.

Logical Flow

Ruby has a number of different looping and conditional flow constructs that control the logical flow of the program through the code. Because these flow statements are usually the most complex individual bits of logic, using consistent style for clarity is important in keeping the code readable and maintainable. Following are a few suggestions.

Even though Ruby has a for keyword, it is better to use each for looping over an enumeration—for is defined in terms of each, so you don't gain speed and you lose some clarity by using for.

In if and when statements, only use then where the condition and the result are on the same line. Only use guard clauses at the end of the line like foo if x.nil? if the clause is simple. If the clause is complex, consider breaking it out into its own method with a meaningful name.

It's fine to use unless instead of if !, but please don't use unless followed by else, that tends to be very confusing.

The if and case statements return a value, and you should use this fact to make code clearer, like this:

```
result = if a_thing
           x
         else
           y
         end
```

Note the indentation (you can also indent relative to the beginning of the line). Also note that Ruby style does not put parentheses around the boolean condition in an if or while statement.

For single-line conditional assignments, both of these styles are correct: result = if a_thing then x else y end and result = a_thing ? x : y. They each have their fans.